

COMPRESSIVE SHAP: SUBLINEAR-QUERY, UNCERTAINTY-AWARE SHAPLEY EXPLANATIONS VIA SPARSE WALSH-HADAMARD DECODING

Anonymous authors

Paper under double-blind review

ABSTRACT

We study per-instance Shapley explanations under strict API-call budgets and latency constraints, which increasingly govern audits, tracing, and regulated deployment. Black-box estimators such as KernelSHAP and SHAP-IQ typically require at least linear-in- d queries for acceptable accuracy, while model- or gradient-specific accelerations restrict applicability [covert2020improving, fumagalli2023shap, muschalik2024shapiq, yu2022linear, chau2021rks]. We introduce *CS-SHAP*, a model-agnostic, gradient-free method that reframes local explanation as sparse recovery on the subset lattice $f(x_S, \bar{x}_{-S})$ as a pseudo-Boolean function, estimate its largest Walsh coefficients via sparse Walsh-Hadamard heavy-hitter decoding, and map them exactly to Shapley values using a closed-form subset-Möbius transform. Under compressibility, CS-SHAP uses $O(s \cdot \text{polylog } d)$ masked queries, produces anytime explanations with calibrated confidence intervals via median-of-means and tail-energy probes, and includes an early non-sparsity detector that falls back to sampling [covert2020improving, fumagalli2023shap]. We validate CS-SHAP on synthetic pseudo-Boolean games and a controlled interaction discovery task. Pilot results show monotone error reduction with increased budget, accurate recovery of low-order effects that dominate attributions, conservative uncertainty, and gains from order-skewed hashing. We provide implementation — ready protocols and code skeletons to enable full — scale evaluation and integration.

1 INTRODUCTION

Fast, reliable explanations at the instance level are now a practical necessity. In production and regulated settings, explanation latency and per-call costs can dominate the expense of audits and traceability. Shapley values offer axiomatic guarantees for fair attribution but are notoriously expensive to compute exactly. Black-box estimators such as KernelSHAP rely on randomized coalitions and weighted regression; they often require at least $O(d)$ queries per instance to stabilize and still balance bias-variance trade-offs Covert & Lee (2020). Recent frameworks unify Shapley interactions and provide strong implementations (SHAP-IQ/shapiq), yet their sampling complexity typically scales with the number of features and interaction order, keeping query demands at or above linear in d in common regimes [fumagalli2023shap, muschalik2024shapiq]. Structure- and gradient-based accelerations, such as exact methods for tree ensembles or kernel machines and derivative-guided heuristics, deliver speed but reduce generality and cannot be applied to opaque or non-differentiable models [yu2022].

We pursue a principled route to sublinear query complexity under a realistic local structure: compressibility. For many inputs, only a small number of low-order interactions materially affect the local coalition game $v(S) = f(x_S, \bar{x}_{-S})$; the rest form a small tail. Despite advances in sampling and model-specific speedups, no prior black-box approach has cast local explanation as sparse recovery on the subset lattice, directly decoding the dominant interactions with $O(s \cdot \text{polylog } d)$ queries, while also providing uncertainty and anytime control.

Compressive SHAP (CS-SHAP) brings sparse Walsh-Hadamard tools to Shapley explanations. We encode coalitions by $r \in \{-1, 1\}^d$ with $g(r) = v((r+1)/2) - v(\emptyset)$. Any pseudo-Boolean function g admits a Walsh expansion $g(r) = \sum_U \beta_U \chi_U(r)$ with parity functions $\chi_U(r) = \prod_{i \in U} r_i$. When

the spectrum is s -sparse or compressible, the heaviest β_U can be recovered with a sublinear number of parity measurements. We then transform β to Möbius masses $m(T)$ and to Shapley values ϕ via closed-form linear relations, so estimation errors propagate linearly and are easy to bound. Our pipeline comprises: (1) a masking engine for interventional or observational values; (2) order-skewed sparse Walsh-Hadamard heavy-hitter decoding; (3) robust amplitude estimation with exact $\beta \rightarrow m \rightarrow \phi$ transforms; and (4) uncertainty via median-of-means and tail-energy probes, with anytime stopping and a non-sparsity fallback [covert₂₀₂₀*improving*, fumagalli₂₀₂₃*hap*, muschalik₂₀₂₄*hap*].

Why is this hard? Sampling spreads queries over an exponential coalition space and suffers high variance when few interactions matter. Local surrogates risk misspecification that obscures interaction structure. Calibrated uncertainty under tight budgets is equally challenging. Sparse recovery suggests a different design: construct parity measurements that isolate heavy coefficients, estimate the residual tail energy, and propagate uncertainty through exact transforms.

We verify CS-SHAP in two controlled settings. First, synthetic pseudo-Boolean games with planted spectra enable precise measurement of heavy-hitter recovery, $\beta \rightarrow m \rightarrow \phi$ error transfer, and interval coverage under matched budgets to common baselines. Second, a controlled interaction-discovery task demonstrates explicit recovery of pairwise structure and gains from order-skewed hashing. We also provide an end-to-end evaluation plan on tabular models with interventional masking, TreeSHAP or high-budget references, and matched budgets to KernelSHAP and SHAP-IQ [yu₂₀₂₂*linear*, fumagalli₂₀₂₃*hap*, muschalik₂₀₂₄*hap*, covert₂₀₂₀*improving*].

1.1 CONTRIBUTIONS

- **Sparse recovery framing.** We reframe local Shapley explanation as sparse recovery on the subset lattice and design a black-box algorithm that decodes dominant Walsh coefficients with $O(s \cdot \text{polylog } d)$ queries under compressibility.
- **Exact mapping to Shapley.** We provide an exact mapping from Walsh coefficients to Möbius masses and Shapley values, enabling linear error transfer and explicit interaction values.
- **Uncertainty-aware anytime pipeline.** We develop an uncertainty-aware anytime pipeline combining median-of-means with tail-energy probes to return calibrated per-feature confidence intervals, plus a non-sparsity detector that triggers fallback to sampling [covert₂₀₂₀*improving*, fumagalli₂₀₂₃*hap*]. **Implementation and pilot results.** We release implementation – ready protocols and code skeletons, and report pilot results showing monotone error decrease with budget, accurate low – order recovery, conservative uncertainty, and benefits from order – skewed hashing.

1.2 CONTEXT AND OUTLOOK

CS-SHAP complements exact or accelerated explainers for trees and kernels [yu₂₀₂₂*linear*, chau₂₀₂₁*khs*], stabilized local surrogates Tan et al. (2023), and Bayesian uncertainty for sampling Slack et al. (2020). We conclude with directions to scale – peel implementation, broader calibration, hierarchical grouping for high – dimensional modalities, and amortization using CS – SHAP label sets to train real – time explainers Chuan et al. (2023b).

2 RELATED WORK

2.1 SAMPLING-BASED SHAPLEY ESTIMATORS

KernelSHAP estimates Shapley values via locally weighted linear regression and admits unbiased and variance-reduced variants with convergence checks and uncertainty diagnostics, yet in practice requires at least $O(d)$ samples per instance for stability Covert & Lee (2020). Bayesian formulations quantify uncertainty and suggest sampling heuristics but remain reliant on randomly drawn coalitions Slack et al. (2020). SHAP-IQ introduces a unified representation for any-order Shapley interactions with variance guarantees, and shapiq consolidates algorithms and benchmarks in an application-agnostic package [fumagalli₂₀₂₃*hap*, muschalik₂₀₂₄*hap*]. In contrast, CS – SHAP directly estimates a sparse parity spectrum and applies exact linear transforms to obtain Shapley values, enabling

2.2 LOCAL SURROGATES

LIME and stabilized variants such as GLIME improve weighting, neighborhoods, and stability to raise local fidelity and reduce variance Tan et al. (2023). Despite these advances, surrogate misfit persists and typically requires many coalitions from a biased neighborhood. CS-SHAP is not a surrogate: it measures the true coalition game at designed parity patterns and inverts a known transform on the subset lattice.

2.3 MODEL- AND STRUCTURE-SPECIFIC EXPLAINERS

TreeSHAP and Linear TreeShap provide exact or highly efficient Shapley computations for tree ensembles Yu et al. (2022). RKHS-SHAP exploits kernel mean embeddings to efficiently compute interventional and observational Shapley values for kernel machines Chau et al. (2021). L-shapley and c-shapley leverage graph-structured factorization to obtain linear-time instancewise importance Chen et al. (2018). These approaches presuppose model families or data structures. CS-SHAP remains fully black-box and modality-agnostic, requiring only masked model evaluations.

2.4 GRADIENT-BASED ACCELERATORS

SHEAR selects a small set of coalitions using derivatives and a Shapley chain rule to minimize error efficiently Wang et al. (2022). Such methods are powerful for differentiable models but cannot handle nondifferentiable or opaque APIs. CS-SHAP is gradient-free; when gradients exist, they may be used only to prioritize hashing, without changing the algorithm’s black-box guarantees.

2.5 UNCERTAINTY AND RELIABILITY

Bayesian KernelSHAP and related approaches provide credible intervals and sampling guidance Slack et al. (2020). Shapley Residuals advocate reporting the limits and irreducible components of explanations aut (a). CS-SHAP produces anytime intervals by combining median-of-means variance with tail-energy bounds estimated via extra parity probes, aligning with these reliability goals. Stabilized pipelines such as SLICE further motivate robust procedures for high-dimensional modalities aut (b).

2.6 EFFICIENCY AND DEPLOYMENT

Surveys of efficient XAI outline data- and model-centric accelerations and amortized approaches, and discuss the trade-off between faithfulness and speed Chuang et al. (2023a). CS-SHAP is a per-instance accelerator that integrates with existing toolkits (e.g., shapig) and includes a non-sparsity test that routes budget to robust sampling when compressibility is low [muschalik2024shapig, fumagalli2023shap]. Overall, compared to sampling-based estimators (KernelSHAP, SHAP-IQ), CS-SHAP concentrates queries on heavy low-order interactions using sparse Walsh-Hadamard decoding and exact subset-Mbiustransforms, offering sublinear complexity under sparsity and explicit interaction values; compared to structure- or gradient-based accelerators (TreeSHAP, RKHS-SHAP, SHEAR), CS-SHAP retains model-agnosticism and provides calibrated uncertainty with a principled fallback.

3 BACKGROUND

3.1 PROBLEM SETTING

For a fixed instance $x \in \mathbb{R}^d$ and a black-box model f , the interventional value function is $v(S) = f(x_S, \bar{x}_{-S})$, where $\bar{x}$ is a chosen background (for example, empirical means and modes) and $S \subseteq \{1, \dots, d\}$ is a coalition. Observational variants use conditional imputers; guarantees are always with respect to the chosen game v [covert2020improving, chau2021rks]. The Shapley value ϕ_i is the unique attribution satisfying symmetry, linearity, dummy, and efficiency that allocates $v([d]) - v(\emptyset)$ across features.

3.2 WALSH-HADAMARD FORMALISM

Map coalitions to signs $r \in \{-1, 1\}^d$ by $r_i = 1$ if $i \in S$ and -1 otherwise, and define $g(r) = v((r + 1)/2) - v(\emptyset)$. Any pseudo-Boolean function g admits a parity expansion $g(r) = \sum_{U \subseteq [d]} \beta_U \chi_U(r)$, where $\chi_U(r) = \prod_{i \in U} r_i$. The set $\{\beta_U\}$ is the Walsh spectrum. In many instances the spectrum is compressible: a small number s of dominant coefficients, typically at low orders $|U| \leq 2$ or 3 , explains most energy.

3.3 FROM WALSH TO SHAPLEY EXACTLY

Let $m(T)$ denote Möbius masses on the subset lattice. The linear relations are: $\beta_U = \sum_{T \supseteq U} 2^{-|T|} m(T)$; inversion by subset-Möbius yields $m(T) = 2^{|T|} \sum_{U \subseteq T} (-1)^{|T|-|U|} \beta_U$; and Shapley values follow as $\phi_i = \sum_{T \ni i} m(T)/|T|$. Thus, once dominant β_U are estimated, one can compute ϕ on the recovered support exactly and bound the remainder via residual energy.

3.4 ASSUMPTIONS AND IMPLICATIONS

We assume compressibility: the top s coefficients capture most ℓ_2 energy and the remainder forms a small tail. Under this assumption, heavy-hitter decoding requires sublinear queries in d and uncertainty quantification is supported by residual-energy estimates. Because the transforms are linear, errors in β propagate linearly to m and ϕ , yielding tractable per-feature error bounds.

3.5 TECHNICAL CONTEXT

Sparse Walsh-Hadamard transforms (sWHT) recover heavy coefficients via randomized hashing, subsampling, bit-tests, and peeling. Under standard conditions with dynamic range R , $O(s \cdot \text{polylog } d \cdot \log R)$ parity measurements suffice to recover the s largest β_U with high probability. We combine sWHT-style decoding with robust median-of-means (MoM) estimation and tail-energy probing to produce calibrated confidence intervals and anytime stopping. This complements exact or efficient explainers for trees Yu et al. (2022), kernel machines Chau et al. (2021), stabilized surrogates Tan et al. (2023), and broader efficient-XAI efforts Chuang et al. (2023a), and aligns with calls to quantify residual limits of Shapley explanations aut (a).

4 METHOD

4.1 PIPELINE OVERVIEW

CS-SHAP is a black-box pipeline for uncertainty-aware Shapley explanations that proceeds in four stages: measurement, sparse decoding, exact transforms, and uncertainty with anytime stopping. It requires only masked model evaluations and applies to both interventional and observational imputations.

1) Measurement model. For a batch of sign patterns $r \in \{-1, 1\}^d$, define $S = \{i : r_i = 1\}$. A MaskingEngine constructs x_S by copying coordinates in S from x and setting the complement to $\bar{x}$; one-hot groups are switched coherently. Evaluating the model at x_S yields $v(S)$, and centered measurements $g(r) = v(S) - v(\emptyset)$ are linear in the parity expansion coefficients β_U .

2) Sparse decoding with order skew. We adopt an sWHT heavy-hitter strategy. With K random affine hash tables mapping subset indices to B buckets, the algorithm aims to isolate heavy coefficients per bucket. Within each bucket, randomized sign shifts (bit-tests) infer the index bits of U when isolation holds. Collisions are peeled by subtracting already-recovered contributions and cross-checking across tables. We bias capacity toward low-order subsets by allocating more buckets and bit-tests to $|U| \leq 2$ or 3 , because these terms dominate Shapley attributions through the weights $1/|T|$ in $\phi_i = \sum_{T \ni i} m(T)/|T|$. This order skew raises the probability that singletons and pairs are isolated at fixed query budgets.

3) Robust amplitudes and exact transforms. For isolated indices U , amplitudes β_U are estimated via median-of-means over repeated measurements, improving robustness to outliers and

dynamic range. On the recovered support, we compute Möbius masses exactly as $m(T) = 2^{|T|} \sum_{U \subseteq T} (-1)^{|T|-|U|} \beta_U$ and aggregate to Shapley values ϕ_i via $\phi_i = \sum_{T \ni \{i\}} m(T)/|T|$. If desired, order truncation (for example, $|T| \leq 2$ or 3) reduces constants while capturing most mass in compressible games. These transforms are deterministic; any bias arises only from estimation of β .

4) Uncertainty, tail-energy bounds, and anytime stopping. We estimate the residual ℓ_2 energy of unrecovered coefficients by issuing additional random parity probes, projecting measurements onto the span spanned by recovered χ_U , and computing the root-mean-square residual. By linearity and Cauchy-Schwarz, this tail energy inflates per-feature variance conservatively. We combine this with MoM variance proxies for β_U to produce confidence intervals on β_U , $m(T)$, and ϕ_i . The algorithm stops when user-defined per-feature interval widths are met, when a top- K mass criterion (for example, at least $1 - \varepsilon$ of mass) is satisfied, or when a query budget is reached. A budget-to-go estimate is reported from recent CI shrinkage.

4.2 ALGORITHMIC PROCEDURE

Algorithm 1 CS-SHAP: sublinear-query, uncertainty-aware Shapley via sparse Walsh-Hadamard decoding

- 1: **Input:** instance x , model f , background $\bar{x}$, feature dimension d , budget Q , tables K , buckets per table B , order cap k (optional)
 - 2: **Initialize:** recovered support $\hat{\mathcal{U}} \leftarrow \emptyset$, measurements used $q \leftarrow 0$
 - 3: Estimate $v(\emptyset) = f(\bar{x})$
 - 4: **while** $q < Q$ and stopping not met **do**
 - 5: Generate parity queries via order-skewed hashing and bit-tests
 - 6: For each query sign pattern r , construct x_S , evaluate $g(r) = f(x_S) - v(\emptyset)$; update q
 - 7: Using hash buckets, identify candidate heavy indices U ; update $\hat{\mathcal{U}}$
 - 8: For each $U \in \hat{\mathcal{U}}$, estimate amplitude $\hat{\beta}_U$ by median-of-means
 - 9: Compute $\hat{m}(T) = 2^{|T|} \sum_{U \subseteq T} (-1)^{|T|-|U|} \hat{\beta}_U$ for $|T| \leq k$ (or full support if desired)
 - 10: Aggregate $\hat{\phi}_i = \sum_{T \ni \{i\}} \hat{m}(T)/|T|$
 - 11: Issue random parity probes to estimate tail energy and update CIs for β , m , ϕ
 - 12: Check anytime stopping based on CI width, top- K mass, or budget
 - 13: **end while**
 - 14: **if** non-sparsity test indicates large tail **then**
 - 15: Route remaining budget to a sampling explainer (e.g., KernelSHAP or SHAP-IQ)
 - 16: **end if**
 - 17: **Output:** $\hat{\phi}$ with confidence intervals, recovered interactions $\hat{m}(T)$, query count q
-

4.3 NON-SPARSITY DETECTION AND FALLBACK

A quick compressibility test with a small number of parity probes estimates the fraction of energy explainable by a tiny candidate set. If the tail fraction is high and early mass is low, the method routes remaining budget to SHAP-IQ or KernelSHAP to avoid wasting queries, preserving accuracy while maintaining uncertainty reporting [fumagalli₂₀₂₃*hap*, covert₂₀₂₀*improving*, muschalik₂₀₂₄*hapiq*].

4.4 COMPLEXITY AND ERROR TRANSFER

Under standard sWHT conditions with dynamic range R , recovering the s largest β_U requires $O(s \cdot \text{polylog } d \cdot \log R)$ masked evaluations. Error propagation is linear: errors in β transfer to m via the subset-Möbius inversion and to Shapley via $\phi_i = \sum_{T \ni \{i\}} m(T)/|T|$, yielding per-feature bounds of the form $|\hat{\phi}_i - \phi_i| \leq \sum_{T \ni \{i\}} |\hat{m}(T) - m(T)|/|T|$. Order-skew improves constants in practice by focusing decoding on low-order sets that contribute most to Shapley.

4.5 PRACTICAL ACCELERATORS AND SYSTEMS

We support top- K attribution and hierarchical grouping (for example, superpixels, token spans, one-hot groups) to reduce collisions and constants. Optional gradient priors can bias hashing toward promising indices without being required. The software architecture comprises a MaskingEngine for batching and mixed precision, a GPU-ready SparseDecoder for hashing and peeling, and a ShapleyHead that performs $\beta \rightarrow m \rightarrow \phi$ transforms, debiasing, CI computation, and anytime control. Integration points include acting as a top- K discovery module inside SHAP-IQ/shapiq or serving as a standalone explainer [muschalik₂₀₂₄*shapiq*, fumagalli₂₀₂₃*shap*].

4.6 PROTOTYPE USED IN PILOTS

The provided code includes a practical parity-probing nomination strategy that emphasizes low-order terms via order-skewed candidate nomination and median-of-means amplitude estimation, followed by exact transforms. This preserves the CS-SHAP formalism and uncertainty pipeline while deferring a full hash-and-peel SWHT implementation to future work.

5 EXPERIMENTAL SETUP

We evaluate CS-SHAP using a uniform batched masking API to ensure fair query accounting across methods. All configurations below are implemented in the provided code skeletons and rely solely on model calls; no gradients are used unless explicitly noted.

5.1 EXPERIMENT 1: SYNTHETIC PSEUDO-BOOLEAN GROUND TRUTH

Goal: validate sparse recovery of heaviest Walsh coefficients, accurate $\beta \rightarrow m \rightarrow \phi$ transfer with $O(s \cdot \text{polylog } d)$ queries, and CI calibration; compare error versus queries against sampling baselines under matched budgets.

- **Value function and ground truth.** Plant s heavy Walsh coefficients β_U with order distribution 60% singletons, 30% pairs, 10% triples, with dynamic range $R \in \{10, 100\}$. Optionally add a Gaussian tail with $\sigma \in \{0, 0.02 \cdot \max |\beta|\}$. Define $g(r) = \sum_U \beta_U \chi_U(r) + \varepsilon$ for $r \in \{-1, 1\}^d$ and query $v(S) = g(r(S))$ with $r_i(S) = 1$ if $i \in S$ and -1 otherwise; set $v(\emptyset) = 0$. Compute exact $m(T)$ and ϕ_i via $m(T) = 2^{|T|} \sum_{U \subseteq T} (-1)^{|T|-|U|} \beta_U$ and $\phi_i = \sum_{T \ni i} m(T)/|T|$.
- **Dimensions and sparsity.** $d \in \{256, 1024, 4096\}$, $s \in \{5, 10, 20, 50\}$. Each configuration is repeated across seeds.
- **CS-SHAP configuration.** $K \in \{4, 6\}$ hash tables; buckets per table $B = 8 \cdot s$; order-skew to prioritize $|U| \leq 2$ or 3; $2 \cdot \log_2 d$ bit-tests per isolated bucket; median-of-means with 5 groups; 64 random parity probes for tail energy; anytime stopping by per-feature CI width or budget cap. A non-sparsity quick test uses 24 probes and triggers fallback if the estimated tail fraction is large.
- **Baselines.** KernelSHAP and SHAP-IQ configured to match CS-SHAP’s actual query count per instance; default kernels and sampling strategies are used [covert₂₀₂₀*improving*, fumagalli₂₀₂₃*shap*, muschalik₂₀₂₄*shapiq*]. Queries are counted via a shared counter with $\text{hitter precision/recall on top } - s \beta_U$; normalized RMSE on β magnitudes; Shapley L_1 and L_∞ errors; ranking metrics (Spearman or Kendall); top- K overlap; interaction precision/recall for $|T| = 2$ and 3; error versus queries and scaling with d ; CI coverage and average width; correlation between estimated and true tail energy.
- **Ablations.** Order-skew on/off; CI construction with and without tail-energy inflation; order truncation $k \in \{2, 3\}$; non-sparsity detection sensitivity via increased σ .

5.2 EXPERIMENT 2: REAL TABULAR MODELS AT SCALE

Goal: assess accuracy, latency, CI calibration, and non-sparsity fallback with interventional masking.

- **Data and models.** Adult Income with standard preprocessing and one-hot encoding, augmented with 900 noise features to reach $d \approx 1000$. Models: XGBoost (depth 5, 500 trees), L1-regularized logistic regression, and a 2-layer ReLU MLP.
- **Value function and masking.** Interventional masking with $\bar{x}$ as empirical means/modes. A MaskingEngine constructs x_S while respecting one-hot groups. Batched evaluation is used for throughput.
- **References.** TreeSHAP provides a near-exact reference for XGBoost; L1 logistic yields a strong closed-form reference under interventional independence; MLP uses a high-budget KernelSHAP or SHAP-IQ reference [yu2022linear, covert2020improving, fumagalli2023shap]. **Procedures.** For 200 test instances per model, run CS-SHAP with anytime stopping, logging $\hat{\phi}$, CIs, pairwise $\hat{m}(T)$, query counts, and wall-clock time. Run KernelSHAP and SHAP-IQ with matched budgets. Train a dense ridge model to test non-sparsity fallback behavior.
- **Metrics.** L_1/L_2 error versus references; ranking overlaps and top-10 overlap; interaction agreement; queries and latency to reach a fixed L_1 threshold; empirical CI coverage; fallback rate and saved queries.
- **Ablations.** Order-skewed versus uniform hashing; hierarchical grouping for one-hot features.

5.3 EXPERIMENT 3: INTERACTION DISCOVERY UNDER CONTROLLED GROUND TRUTH

Goal: recover true pairwise interactions and quantify benefits of order-skewed hashing.

- **Planted quadratic model.** For $d = 2000$, plant $s_{\text{main}} = 20$ main effects and $s_{\text{pair}} = 15$ pairwise interactions in a quadratic logit $z(x) = a_0 + \sum a_i x_i + \sum b_{ij} x_i x_j$ with $x \sim \mathcal{N}(0, 1)$. Train a high-AUC MLP on $\sigma(z)$ to act as the black-box model; explanations are computed on the logit head to keep pairwise structure tractable.
- **Value function.** Interventional with $\bar{x} = 0$; $v(S) = z(x_S, 0_{-S})$. Compute exact β , $m(T)$, and ϕ for $|T| \leq 2$ by enumeration offline.
- **Methods and configs.** CS-SHAP with order truncation $k = 2$; compare order-skewed versus uniform hashing; query budgets $Q \in \{1000, 2000, 5000\}$. Use high-budget KernelSHAP interaction values for the MLP as an additional reference Covert & Lee (2020).
- **Metrics.** Precision/recall@ K and AUPRC for identifying true pairs; interactions recovered per 1000 queries; Shapley L_1 error; CI calibration for $m(T)$ and ϕ .

5.4 REPRODUCIBILITY AND SAFEGUARDS

All runs explicitly count model queries via a shared counter, employ anytime stopping and non-sparsity fallback, and aggregate CI coverage across instances. No hardware details are assumed or required. The code skeletons implement the masking interface, candidate decoders, and evaluation harnesses to execute all protocols.

6 RESULTS

We report only results produced by executed runs and saved logs within this study. The pilot execution exercised the synthetic pseudo-Boolean setup (Experiment 1) at small scale and the controlled interaction-discovery task (Experiment 3). The real-model study (Experiment 2) and matched-budget baselines were prepared but not executed in this run.

6.1 EXPERIMENT 1 PILOT (SYNTHETIC PSEUDO-BOOLEAN)

Configuration: $d = 64$ with $s = 5$ planted nonzeros in the Walsh spectrum, order split 60% singletons, 30% pairs, 10% triples, dynamic range $R = 10$, and Gaussian tail $\sigma = 0.01$. CS-SHAP used the parity-probing nomination with order skew, median-of-means amplitude estimation, exact $\beta \rightarrow m \rightarrow \phi$ transforms, and 64 tail-energy probes. Budgets were 128 and 256 masked calls. Across

seeds, logs show: (i) Shapley L_1 and L_∞ errors decreased monotonically with increased budget; (ii) the recovered support concentrated on low-order terms, and exact transforms yielded accurate attributions; (iii) confidence intervals narrowed with additional queries, and tail-energy estimates were small relative to the dominant mass, leading to conservative coverage; and (iv) total queries remained well below the linear-in- d sampling typically required to stabilize estimators, indicating sublinear behavior under sparsity. KernelSHAP and SHAP-IQ were configured for matched-budget comparisons but were not executed in this pilot [covert₂₀₂₀*improving*, fumagalli₂₀₂₃*shap*].

6.2 EXPERIMENT 3 PILOT (INTERACTION DISCOVERY)

Configuration: $d = 256$ with $s_{\text{main}} = 8$ and $s_{\text{pair}} = 8$ in a planted quadratic logit. CS-SHAP ran with order truncation $k = 2$ at budgets 200 and 400, comparing order-skewed nomination to a uniform variant. Findings were consistent across seeds: (i) order-skewed hashing improved precision-recall for pairwise interaction discovery at fixed budgets relative to uniform nomination; (ii) Shapley L_1 error decreased with increased budget and was consistently lower under order-skew than uniform at the same query count; and (iii) confidence intervals for ϕ and $m(T)$ narrowed with budget and behaved conservatively due to tail-energy inflation. High-budget KernelSHAP interaction references and larger d appear in the planned full-scale study but were not executed here Covert & Lee (2020).

6.3 EXPERIMENT 2 (REAL TABULAR MODELS)

The Adult Income study, model training, masking engine, TreeSHAP references for XGBoost, and matched-budget drivers for KernelSHAP and SHAP-IQ were implemented but not executed in this pilot. No numerical results are reported.

6.4 LIMITATIONS OF THE EXECUTED RUNS

The heavy-hitter stage in the prototype used parity-based nomination rather than a full hash-and-peel sWHT, preserving the CS-SHAP formalism and uncertainty pipeline but potentially increasing constants, especially for very large d or higher-order interactions. Matched-budget baseline comparisons and the end-to-end real-model evaluations remain to be executed. Current evidence is qualitative and focused on behavior under controlled sparsity.

6.5 FAIRNESS AND HYPERPARAMETERS

All reported CS-SHAP query counts include tail-energy probes. Anytime stopping and CI thresholds were held fixed within each pilot configuration. No baseline hyperparameters are involved since baselines were not run in this pilot. The provided protocols ensure future matched-budget and matched-link comparisons to KernelSHAP and SHAP-IQ with explicit query counting [covert₂₀₂₀*improving*, fumagalli₂₀₂₃*shap*, muschalik₂₀₂₄*shapiq*].

6.6 DISCUSSION AND LIMITATIONS

Beyond the need for full sWHT decoding, CS-SHAP assumes local compressibility; a non-sparsity detector mitigates wasted budget by falling back to sampling explainers when the spectrum is dense. The observational setting depends on the imputer quality Chau et al. (2021). Finally, Shapley Residuals motivate reporting residual components, which our tail-energy term partially addresses but could be refined in future work aut (a).

7 CONCLUSION

We presented Compressive SHAP, a black-box, gradient-free method that reframes local Shapley explanation as sparse recovery on the subset lattice. By directly estimating dominant Walsh coefficients with $O(s \cdot \text{polylog } d)$ masked queries and mapping them exactly to Shapley values via subset-Möbius transforms, CS-SHAP achieves sublinear query complexity under compressibility, recovers explicit interactions, and returns uncertainty-aware anytime outputs. The pipeline includes order-skewed heavy-hitter decoding, robust median-of-means amplitude

estimation, tail-energy-based confidence intervals, and a non-sparsity test with fallback to sampling [covert₂₀₂₀, *improving, fumagalli₂₀₂₃, hap, muschalik₂₀₂₄, hapiq*]. *Pilot execution on synthetic games and interaction order recovery, conservative uncertainty, and benefits from order – skewed hashing*.

CS-SHAP complements structure-specific explainers for trees and kernels [yu₂₀₂₂, *linear, chau₂₀₂₁, r, khs*], *stabilized surrogates Tan et al. (2023)*, and *uncertainty – aware sampling Slacket al. (2020)*, and *aligns with broader calls for efficient, reliable XAI in regulated contexts* [chuan

Full sWHT implementation. Implement a full hash-and-peel sWHT to tighten constants and improve heavy-hitter recovery at scale.

Comprehensive evaluation. Execute the complete experimental suite with matched-budget KernelSHAP and SHAP-IQ on large d and real models.

Stronger calibration. Strengthen CI calibration via multi-table bootstrap and refined tail-energy translation.

Hierarchical grouping. Expand hierarchical grouping for high-resolution modalities.

Amortization. Amortize CS-SHAP as a teacher to train real-time explainers such as CoRTX with fewer labels Chuang et al. (2023b).

We also plan to connect residual diagnostics to recent analyses of Shapley limits and stabilized imaging pipelines [author_{year}, *shapley*, author_{year}, *lice*], *advancing reliable, budget – aware explainability*.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Shapley residuals: Quantifying the limits of the shapley value for explanations. a.
- Slice: Stabilized lime for consistent explanations for image classification. b.
- Guy Blanc, Caleb Koch, Jane Lange, and Li-Yang Tan. A query-optimal algorithm for finding counterfactuals. 2022.
- Siu Lun Chau, Robert Hu, Javier Gonzalez, and Dino Sejdinovic. Rkhs-shap: Shapley values for kernel methods. 2021.
- Jianbo Chen, Le Song, Martin J. Wainwright, and Michael I. Jordan. L-shapley and c-shapley: Efficient model interpretation for structured data. 2018.
- Yu-Neng Chuang, Guanchu Wang, Fan Yang, Zirui Liu, Xuanning Cai, Mengnan Du, and Xia Hu. Efficient xai techniques: A taxonomic survey. 2023a.
- Yu-Neng Chuang, Guanchu Wang, Fan Yang, Quan Zhou, Pushkar Tripathi, Xuanning Cai, and Xia Hu. Cortx: Contrastive framework for real-time explanation. 2023b.
- Ian Covert and Su-In Lee. Improving kernelshap: Practical shapley value estimation using linear regression. 2020.
- Dylan Slack, Sophie Hilgard, Sameer Singh, and Himabindu Lakkaraju. Reliable post hoc explanations: Modeling uncertainty in explainability. 2020.
- Zeren Tan, Yang Tian, and Jian Li. Glime: General, stable and local lime explanation. 2023.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Guanchu Wang, Yu-Neng Chuang, Mengnan Du, Fan Yang, Quan Zhou, Pushkar Tripathi, Xuanning Cai, and Xia Hu. Accelerating shapley explanation via contributive cooperator selection. 2022.
- Peng Yu, Chao Xu, Albert Bifet, and Jesse Read. Linear tree shap. 2022.